PROGRAMMING WITH PYTHON
SUBJECT CODE : 202044504

DEPARTMENT OF INFORMATION TECHNOLOGY
SEMESTER 5

NAME : Rana Riyancee
ENROLLMENT NO. : 124020806011119
BATCH :IT_B (2B8)
ACADEMIC YEAR : 2026-27

Github Link :
https://github.com/riyanceerana-art/124020806011119_Python_Assignment/tree/main/Assignment2

Riyancee Rana (12402080601119)

PROGRAMMING WITH PYTHON(202044504)
A.D.PATEL INSTITUTE OF TECHNOLOGY(INFORMATION TECHNOLOGY)

ASSIGNMENT-2

PROBLEM STATEMENT 1: Given a large text file containing noisy text, extract all valid email addresses whose top-level domain is com, edu or org. Username may contain letters, digits, plus, dot, underscore or hyphen. For each domain, print the number of unique users and the lexicographically smallest email. Invalid emails must be ignored.

CODE :

```
import re from collections import
defaultdict

def main():
    filename = input("Enter text file path: ").strip()
    pattern = re.compile(r'(?<![\w.+-])[\w.+-]+@[\w.-]+\.(?:com|edu|org)\b')
    emails = defaultdict(set)

    try: with open(filename, encoding="utf-8") as
        f:
            for line in f:
                for email in pattern.findall(line):
                    domain = email.split("@", 1)[1]
                    emails[domain].add(email)

        for domain in sorted(emails):
            values = sorted(emails[domain]) print(domain,
            len(values), values[0])

    except FileNotFoundError:
        print("File not found.")
```

if name == " main ": main()

OUTPUT :

```
Enter text file path: emails.txt
adit.edu 1 a.b+1@adit.edu
x.org 1 test-user@x.org
```

PROBLEM STATEMENT 2: You are given multiple saved HTML pages of product listings. Extract product name, price and rating from each page using regular expressions or an HTML

parser. Normalize prices, remove duplicate products by name, and print the top K products with highest rating; ties are broken by lower price and then lexicographic product name.

CODE :

```
import re import
os

def read_products(filename):
    with open(filename, encoding="utf-8") as f: html
        = f.read()

    pattern = re.compile( r'<h2[^>]*>(.*?)</h2>.*?'
        r'class=["\'][^"\']*price[^"\']*["\'][^>]*>(.*?)</[^>]+>.*
        ?'
        r'class=["\'][^"\']*rating[^"\']*["\'][^>]*>(.*?)</[^>]+>',
        re.I | re.S
    )

    products = [] for name, price, rating in
    pattern.findall(html): name =
    re.sub(r"<[^>]+>", "", name).strip() price =
    float(re.sub(r"[^\d.]", "", price)) rating =
    float(re.sub(r"[^\d.]", "", rating))
    products.append((name, price, rating))
    return products
def main(): try:
        p, k = map(int, input().split())
        products = {}
```

```
for _ in range(p):
    filename = input().strip() if
    not os.path.isfile(filename):
        print("File not found:", filename) return

    for name, price, rating in read_products(filename):
        if name not in products or (rating, -price) > (
        products[name][1], -products[name][0]
        ):
            products[name] = (price, rating)
```



```
    result = sorted(
        products.items(), key=lambda x: (-
        x[1][1], x[1][0], x[0])
    )[:k]

    for name, (price, rating) in result: price =
        int(price) if price.is_integer() else price
        print(name, price, rating)

except ValueError:
    print("Invalid input.")

if name == " main ": main()
```

OUTPUT :

```
2 2
page1.html
page2.html
Python Book 499 4.8
C++ Book 599 4.8
```

PROBLEM STATEMENT 3: Create a Python program that connects to a MySQL database, creates Student and Course Registration tables if they do not exist, inserts records from CSV files, and displays students registered for a given course with SPI greater than a threshold. Use parameterized queries and handle database exceptions.

CODE :

```
import csv
import mysql.connector

def connect(): return
    mysql.connector.connect(
    host="localhost",
        user="root",
        password="YOUR_PASSWORD",
        database="college"
    )

def
    create_tables(conn):
```

Riyancee Rana (12402080601119)





```
cur =
    conn.cursor()
    cur.execute("""
        CREATE TABLE IF NOT EXISTS Student(
            student_id INT PRIMARY KEY, name
            VARCHAR(100) NOT NULL,
            spi DECIMAL(3,2) NOT NULL
        ) """)
    cur.execute(""
    "
        CREATE TABLE IF NOT EXISTS CourseRegistration(
            student_id INT, course_id VARCHAR(20),
            FOREIGN KEY(student_id) REFERENCES Student(student_id)
        ) """)
    conn.commit(
    )
    cur.close()

def insert_students(conn, filename):
    cur = conn.cursor() with open(filename, newline="",
    encoding="utf-8") as f:
        for row in csv.DictReader(f):
            cur.execute(
                "INSERT IGNORE INTO Student VALUES (%s,%s,%s)",
                (row["student_id"], row["name"], row["spi"])
```

```
        )
    conn.commit()
    cur.close()

def insert_registrations(conn, filename): cur =
    conn.cursor() with open(filename, newline="",
    encoding="utf-8") as f:
        for row in csv.DictReader(f): cur.execute(
            "INSERT INTO CourseRegistration VALUES (%s,%s)",
            (row["student_id"], row["course_id"])
        )
    conn.commit()
    cur.close()

def show_students(conn, course_id, threshold):
    cur = conn.cursor() cur.execute("""
        SELECT s.student_id, s.name, s.spi, r.course_id
```

Riyancee Rana (12402080601119)



```
FROM
        Student s
        JOIN CourseRegistration r ON s.student_id = r.student_id
        WHERE r.course_id = %s AND s.spi > %s
        ORDER BY s.spi DESC, s.student_id ASC
    """, (course_id, threshold))

    for row in cur:
        print(*row)
    cur.close()
def main():
    student_file = input("Student CSV: ").strip()
    registration_file = input("Registration CSV: ").strip()
    course_id = input("Course ID: ").strip()

    try:
        threshold = float(input("SPI threshold: "))
        if not 0 <= threshold <= 10: raise
        ValueError

        conn = connect()
        create_tables(conn)
        insert_students(conn,
        student_file)
```

```
        insert_registrations(conn,
        registration_file)
        show_students(conn,
        course_id, threshold)
        conn.close()

    except (ValueError, mysql.connector.Error, FileNotFoundError) as e:
        print("Error:", e)

if name == " main ": main()
```

OUTPUT :



Riyancee Rana (12402080601119)



PROBLEM STATEMENT 4: Build a command-line query builder that receives filter conditions and generates safe parameterized SQL for joining Student, Course and CourseRegistration tables. It must support WHERE, ORDER BY, LIMIT and selected projection fields. Reject invalid columns and unsafe input before executing.

CODE :

```
import mysql.connector
ALLOWED_COLUMNS = {
    "student.id": "s.student_id",
    "student.name": "s.name", "student.spi":
    "s.spi",
    "course.id": "c.course_id",
    "course.name": "c.course_name"
}

def build_query(columns, filters, order_column, direction, limit):
    if not columns or any(c not in ALLOWED_COLUMNS for c in columns):
        raise ValueError("Invalid column")

    if order_column not in ALLOWED_COLUMNS: raise
        ValueError("Invalid order column")
```

```
    if direction.upper() not in ("ASC", "DESC"):
        raise ValueError("Invalid sort direction")

    if not 1 <= limit <= 1000:
        raise ValueError("Invalid limit")

    selected = ", ".join(ALLOWED_COLUMNS[c] for c in columns) query
    = f"""
        SELECT
        {selected} FROM
        Student s
        JOIN CourseRegistration r ON s.student_id = r.student_id
        JOIN Course c ON r.course_id = c.course_id """

    params = [] if filters: conditions = [] for
    column, operator, value in filters:
            if column not in ALLOWED_COLUMNS: raise
                ValueError("Invalid filter column")
```

Riyancee Rana (12402080601119)

```
if operator
            not in ("=", ">", "<", ">=", "<=", "LIKE"): raise
                ValueError("Invalid operator")
            conditions.append(f"{ALLOWED_COLUMNS[column]} {operator} %s")
        params.append(value) query += " WHERE " + " AND ".join(conditions)

    query += f" ORDER BY {ALLOWED_COLUMNS[order_column]} {direction.upper()}"
    query += " LIMIT %s" params.append(limit) return query, params

def
    main():
    try:
        columns = input("Columns: ").split(",") columns
        = [c.strip() for c in columns]

        filters = [] text = input("Filter (column,operator,value) or
        blank: ").strip()

        if text:
            column, operator, value = map(str.strip, text.split(",", 2))
            filters.append((column, operator, value))
```

```
        order_column = input("Order column: ").strip()
        direction = input("Direction: ").strip() limit
        = int(input("Limit: "))

        query, params = build_query( columns, filters,
            order_column, direction, limit
        )

        print("SQL_OK")
        print(query)
        print("Parameters:", params)

        conn =
            mysql.connector.connect(
            host="localhost", user="root",
            password="YOUR_PASSWO
            RD",database="college"
        )
        cur = conn.cursor() cur.execute(query,
        params)
```

Riyancee Rana (12402080601119)

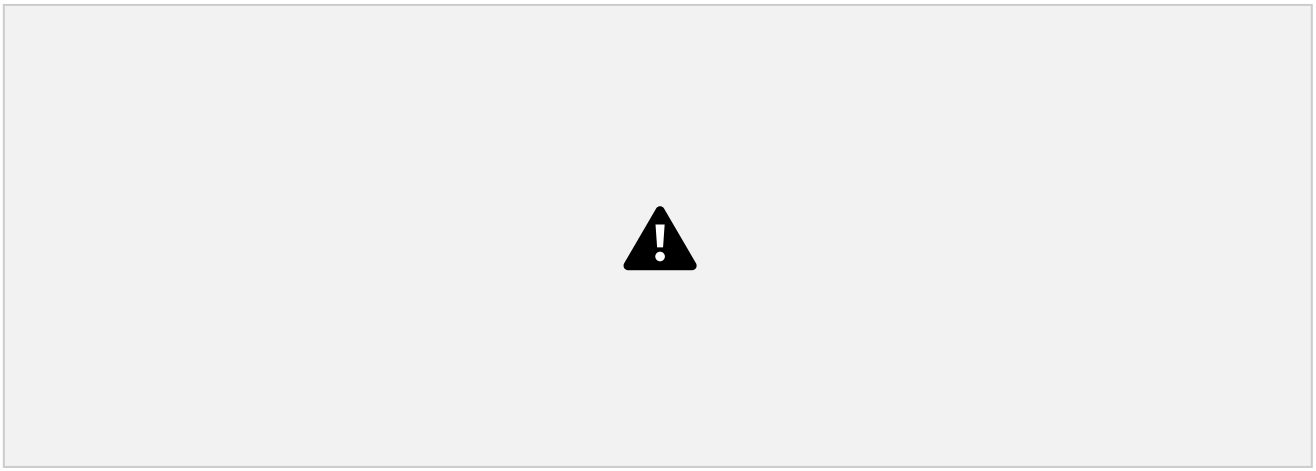

```
for row in
        cur:
            print(*row)

        cur.close() conn.close()

    except (ValueError, mysql.connector.Error) as e:
        print("Error:", e)

if name == " main ": main()
```

OUTPUT :



PROBLEM STATEMENT 5: Develop a Tkinter contact manager that stores contacts in MySQL. It must support create, update, delete, search by name/email/phone, and sorted display. Use Entry, Text, Radio Button or Check Button, Listbox/Menu and Combobox widgets. Validate email using regex before insert/update.

CODE :

```
import tkinter as tk
from tkinter import ttk, messagebox
import sqlite3 import re

def connect():
    return sqlite3.connect("contacts.db")

def setup():
    conn = connect()
    conn.execute("""
```

Riyancee Rana (12402080601119)



```
CREATE TABLE IF NOT EXISTS Contact( id
    INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL, email TEXT UNIQUE
    NOT NULL,
    phone TEXT, category
     TEXT,
    notes TEXT
  ) """)
  conn.commit(
```

```
    )
    return conn

class App:
    def init (self, root): self.root =
        root self.root.title("Contact
        Manager") self.conn = setup()

        labels = ["Name", "Email", "Phone", "Category", "Notes"]
        self.entries = {}

        for i, label in enumerate(labels):
            tk.Label(root, text=label).grid(row=i, column=0, padx=5, pady=5)
           if label == "Category":
               entry = ttk.Combobox( root,
                   values=["Faculty", "Student", "Friend", "Other"],
                   width=32
               ) else: entry = tk.Entry(root,
            width=35)

            entry.grid(row=i, column=1, padx=5, pady=5)
            self.entries[label] = entry

        tk.Button(root, text="Add", command=self.add).grid(row=5, column=0, padx=5)
        tk.Button(root, text="Update", command=self.update).grid(row=5, column=1, padx=5)
        tk.Button(root, text="Delete", command=self.delete).grid(row=5, column=2, padx=5)
        tk.Button(root, text="Search", command=self.search).grid(row=5, column=3, padx=5)
        tk.Button(root, text="Show All", command=self.show_all).grid(row=5, column=4, padx=5)

        self.listbox = tk.Listbox(root, width=100, height=15)
        self.listbox.grid(row=6, column=0, columnspan=5, padx=5, pady=10) self.show_all()
    def get_values(self): return (
        self.entries["Name"].get().strip(),
```

Riyancee Rana (12402080601119)



```
        self.entries["Email"].get().strip(),
        self.entries["Phone"].get().strip(),
        self.entries["Category"].get().strip(),
            self.entries["Notes"].get().strip()
        )

    def valid_email(self, email):
```

```
        return re.fullmatch(r"[\w.+-]+@[\w.-]+\.[A-Za-z]{2,}",
        email)

    def add(self):
        name, email, phone, category, notes = self.get_values()

        if not name or not email:
            messagebox.showerror("Error", "Name and Email are required") return
    if not self.valid_email(email):
            messagebox.showerror("Error", "Invalid email") return

        try:
            self.conn.execute(
                "INSERT INTO Contact(name,email,phone,category,notes) VALUES(?,?,?,?,?)",
                (name, email, phone, category, notes)
            )
            self.conn.commit()
            self.clear()
            self.show_all()
            messagebox.showinfo("Success", "Contact added")
        except sqlite3.IntegrityError: messagebox.showerror("Error",
            "Email already exists")

    def update(self):
        selected = self.listbox.curselection()

        if not selected:
            messagebox.showerror("Error", "Select a contact") return
        name, email, phone, category, notes = self.get_values()
        if not self.valid_email(email):
        messagebox.showerror("Error", "Invalid email")
        return
```

Riyancee Rana (12402080601119)



```
contact_id
        = self.listbox.get(selected[0]).split("|")[0].strip()
        try:
            self.conn.execute(
                """UPDATE Contact
                   SET name=?, email=?, phone=?, category=?, notes=?
                   WHERE id=?""",
```

```
            (name, email, phone, category, notes, contact_id)
        )
        self.conn.commit()
        self.clear()
        self.show_all()
        messagebox.showinfo("Success", "Contact updated")
    except sqlite3.IntegrityError: messagebox.showerror("Error",
        "Email already exists")
def delete(self):
    selected = self.listbox.curselection()

    if not selected:
        messagebox.showerror("Error", "Select a contact") return
    contact_id = self.listbox.get(selected[0]).split("|")[0].strip()
    self.conn.execute(
        "DELETE FROM Contact WHERE id=?",
        (contact_id,)
    )
    self.conn.commit()

    self.clear()
    self.show_all()
    messagebox.showinfo("Success", "Contact deleted")

def search(self):
    term = self.entries["Name"].get().strip()

    rows = self.conn.execute(
        """SELECT id,name,email,phone,category
           FROM Contact
           WHERE name LIKE ? OR email LIKE ? OR phone LIKE ? ORDER
           BY name""",
        (f"%{term}%", f"%{term}%", f"%{term}%")
```

Riyancee Rana (12402080601119)




```
).fetchall()

    self.display(rows)

def show_all(self):
    rows = self.conn.execute(
```

```
            """SELECT id,name,email,phone,category
              FROM Contact
              ORDER BY name"""

        ).fetchall()

        self.display(rows)


    def display(self, rows): self.listbox.delete(0, tk.END)
        for row in rows: self.listbox.insert(tk.END, " |
        ".join(map(str, row)))

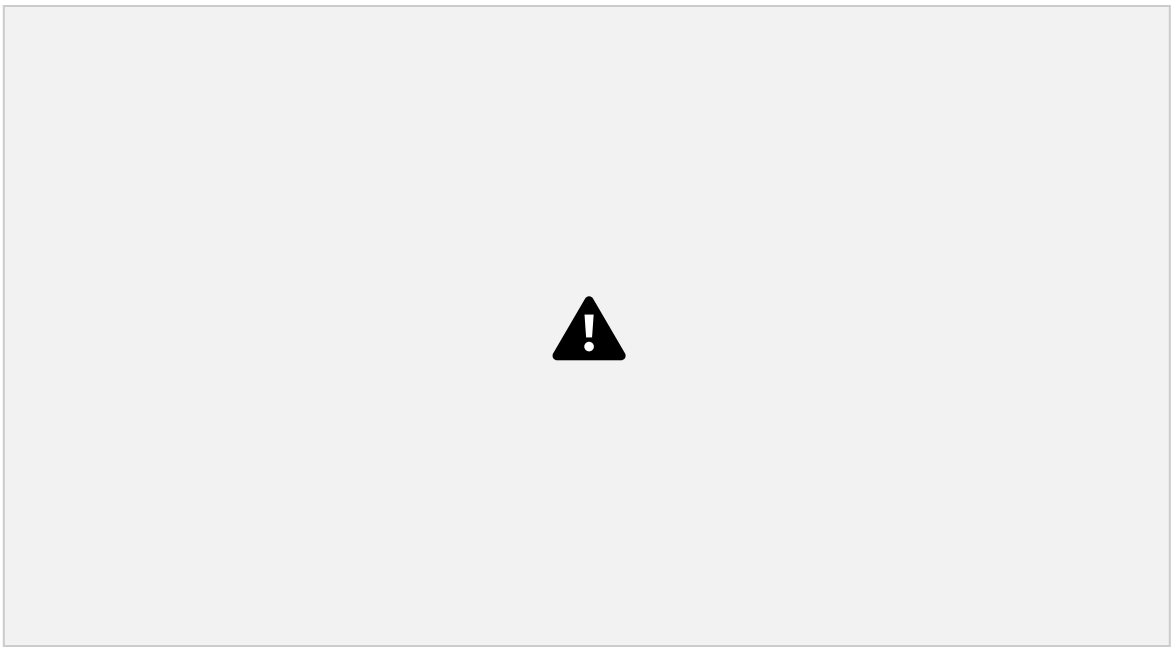
    def clear(self):
        for entry in self.entries.values():
            entry.delete(0, tk.END)


root = tk.Tk()
App(root)
root.mainloop()
```

OUTPUT :

Riyancee Rana (12402080601119)





PROBLEM STATEMENT 6: Given timestamped application logs, detect every user who has at least X failed login attempts within any T-minute window followed by a successful login from a different IP address. Use regex to parse log lines and an optimized sliding window per user. **CODE :**

```
import re from collections import
defaultdict, deque

def to_minutes(t): h, m =
   map(int, t.split(":")) return
   h * 60 + m

def
   main():
   try:
      X, T = map(int, input().split()) n
      = int(input())

      failures = defaultdict(deque) suspicious
      = {}

      pattern = re.compile(
         r"^(\d{2}:\d{2})\s+(\S+)\s+(\S+)\s+(FAIL|SUCCESS)$"
      )
```




for _ in

```
        range(n): line =
            input().strip() match =
            pattern.match(line)

            if not match:
                continue

            timestamp, user, ip, status = match.groups()
            current = to_minutes(timestamp)

            if status == "FAIL":
                q = failures[user]

                while q and current - q[0][0] > T:
                    q.popleft()

                q.append((current, ip))
            else:q = failures[user]

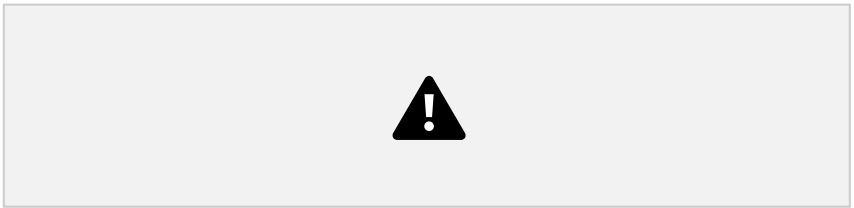
                while q and current - q[0][0] > T:
                    q.popleft()
                if len(q) >= X and any(old_ip != ip for _, old_ip in q):
                    suspicious.setdefault(user, timestamp)

        for user in sorted(suspicious):
            print(user, suspicious[user])

    except ValueError: print("Invalid
        input.")

if name == " main ": main()
```

OUTPUT :



PROBLEM STATEMENT 7: Create an ETL pipeline using generator functions that reads a large JSON-lines file of sensor records, filters corrupted records, transforms units, groups readings by device and produces summary statistics. The program must not load the full file into memory.

 **CODE :**

```
import json from collections
import defaultdict

def read_records(filename):
    with open(filename, encoding="utf-8") as f:
        for line in f: try:
                yield json.loads(line)
            except json.JSONDecodeError:
                yield None

def main():
    filename = input("Enter JSONL file path: ").strip() data
    = defaultdict(lambda: {
        "count": 0,
        "min": float("inf"),
        "max": float("-inf"),
        "sum": 0.0,
        "corrupted": 0
    })

    try:
        for record in
            read_records(filename): if not
            isinstance(record, dict):
            continue

            device = record.get("device_id")
            temperature = record.get("temperature_c")

            if not device or not isinstance(temperature, (int, float)):
                if device:
                    data[device]["corrupted"] += 1
                continue

            d = data[device]

            d["count"] += 1
            d["min"] = min(d["min"], temperature)
            d["max"] = max(d["max"], temperature) d["sum"] += temperature
```

```
    for device in sorted(data):
        d = data[device] if
        d["count"]:
            avg = d["sum"] / d["count"]
            print(
                f"{device} count={d['count']} " f"min={d['min']}
                max={d['max']} "
                f"avg={avg:.2f} corrupted={d['corrupted']}"
            ) else: print(f"{device} count=0
        corrupted={d['corrupted']}")

    except FileNotFoundError:
        print("File not found.")

if name == " main ": main()
```

OUTPUT :



PROBLEM STATEMENT 8: Given an n x m matrix of integers, find the maximum score path from top-left to bottom-right. You may move right, down or diagonally down-right. Some cells are blocked. Also print the number of distinct maximum-score paths modulo 1000000007.

CODE :

```
MOD = 1000000007 NEG
= -10**30
def
    main():
    try:
        n, m = map(int, input().split()) matrix
        = []

        for _ in range(n): row
            = input().split()

            if len(row) != m:
                print("Invalid
```

matrix.") return

Riyancee Rana (12402080601119)



```
    matrix.append([
        None if x == "X" else int(x) for
        x in row
    ])

if matrix[0][0] is None or matrix[n - 1][m - 1] is None:
    print("IMPOSSIBLE")
    return

score = [NEG] * m
ways = [0] * m

for i in range(n):
    new_score = [NEG] * m new_ways
    = [0] * m

    for j in range(m):
        if matrix[i][j] is None: continue
        if i == 0 and j == 0:

            new_score[j] = matrix[i][j] new_ways[j]
            = 1
            continue
        best = NEG
        count = 0

        for pi, pj in ((i - 1, j), (i, j - 1), (i - 1, j - 1)):
            if pi < 0 or pj < 0: continue
            s = score[pj] if pi == i - 1 else new_score[pj] w
            = ways[pj] if pi == i - 1 else new_ways[pj]

            if s > best:
                best = s
                count =
                w
            elif s == best:
                count = (count + w) % MOD

        if best != NEG:
```

new_score[j] = best + matrix[i][j] new_ways[j]
= count

Riyancee Rana (12402080601119)

 score, ways
= new_score, new_ways
if score[m - 1] == NEG:
print("IMPOSSIBLE")
else:
print(score[m - 1], ways[m - 1] % MOD)
except ValueError:
print("Invalid input.")

if name == " main ": main()

OUTPUT :



PROBLEM STATEMENT 9: Design Product and Inventory classes. Inventory must support adding products, deleting products, updating stock, comparing inventories by total value, and merging two inventories using operator overloading. Conflicting product ids must merge stock quantities but preserve the lower purchase price and higher selling price for reporting.

CODE :

class Product:
def init (self, pid, name, stock, purchase, selling): self.pid
= pid self.name = name
self.stock = stock
self.purchase = purchase
self.selling = selling

def value(self): return self.stock
* self.selling

class Inventory:
def init (self):
self.products = {}

```
    def add(self, product):
        self.products[product.pid] = product
```

Riyancee Rana (12402080601119)



def

```
    delete(self, pid):
        self.products.pop(pid,
        None)

    def update(self, pid, stock):
        if pid in self.products: self.products[pid].stock
            = stock

    def total_value(self):
        return sum(p.value() for p in self.products.values())

    def add (self, other): result
        = Inventory()

        for p in self.products.values(): result.add(
                Product(
                    p.pid, p.name, p.stock,
                    p.purchase, p.selling)
            )

        for p in other.products.values():
            if p.pid in result.products: q
                = result.products[p.pid]
                q.stock += p.stock
                q.purchase = min(q.purchase, p.purchase)
                q.selling = max(q.selling, p.selling)
            else:
                result.add(
                    Product(
                        p.pid, p.name, p.stock,
                        p.purchase, p.selling
                    )
                )
```

```
        return result

    def gt (self, other): return self.total_value() >
        other.total_value()

    def display(self):
```

Riyancee Rana (12402080601119)



```
        sorted(self.products.values(), key=lambda x: x.pid):
            print(
                f"{p.pid} {p.name} stock={p.stock} "
                f"purchase={p.purchase} selling={p.selling}"
            )


def main():
    a = Inventory() b = Inventory() n =
    int(input("Products in A: ")) for _ in range(n): pid,
    name, stock, purchase, selling = input().split()
    a.add(Product(
            pid, name, int(stock),
            float(purchase),
            float(selling)
        ))

    n = int(input("Products in B: ")) for _ in range(n): pid,
    name, stock, purchase, selling = input().split()
    b.add(Product( pid, name, int(stock),
    float(purchase),
            float(selling)
        ))

    while True: command = input("Operation:
        ").strip().split()

        if not command:
            continue

        if command[0] == "ADD":
            target = a if command[1] == "A" else b
            target.add(
```

```
            Product(
                command[2],
                command[3],
                int(command[4]),
                float(command[5]),
                float(command[6])
            ) )

    elif command[0] == "DELETE":
```

Riyancee Rana (12402080601119)

```
target = a if command[1] == "A" else b
        target.delete(command[2])

    elif command[0] == "UPDATE":
        target = a if command[1] == "A" else b
        target.update(command[2], int(command[3]))

    elif command[0] == "MERGE":
        merged = a + b merged.display()
        print("TOTAL",
        merged.total_value())

    elif command[0] == "COMPARE": if
        a.total_value() >
    b.total_value():
            print("A")
        elif b.total_value() > a.total_value():print("B") else:
            print("EQUAL")

    elif command[0] == "EXIT":
break if name == " m ain " :
main()
```

OUTPUT :



Riyancee Rana (12402080601119)



PROBLEM STATEMENT 10: Create a Python program that reads a CSV dataset of student marks, cleans missing values, calculates subject-wise statistics and exports at least three visualizations: grade distribution, subject-wise average comparison and top performer chart. The program may use matplotlib/pandas if permitted by faculty and must save charts as image files.

CODE :

```
import os import
pandas as pd
import matplotlib.pyplot as plt

def main():
    filename = input("CSV file path: ").strip()
    output = input("Output folder: ").strip()
    try:
        os.makedirs(output, exist_ok=True)

        df = pd.read_csv(filename) subject_cols
        = list(df.columns[2:])

        for col in subject_cols: df[col] =
            pd.to_numeric(df[col], errors="coerce") df[col] =
            df[col].fillna(df[col].mean())

        df.to_csv( os.path.join(output,
            "cleaned_marks.csv"), index=False
        )

        summary = pd.DataFrame({
```

```
    "subject": subject_cols,
    "average": [df[c].mean() for c in subject_cols],
    "minimum": [df[c].min() for c in subject_cols],
    "maximum": [df[c].max() for c in subject_cols]
})

summary.to_csv( os.path.join(output,
    "summary.csv"), index=False
)
grades = pd.cut(df[subject_cols].mean(axis=1), bins=[-1, 40, 50, 60, 70, 80, 90, 100],
    labels=["F", "D", "C", "B", "A", "A+", "O"]
)
```

Riyancee Rana (12402080601119)



```
grades.value_counts().sort_index().plot(kind="bar"
) plt.xlabel("Grade") plt.ylabel("Students")
plt.tight_layout() plt.savefig(os.path.join(output,
"grade_distribution.png")) plt.close()

summary.plot(
    x="subject",
    y="average",
    kind="bar",
    legend=False
)
plt.ylabel("Average Marks")
plt.tight_layout()
plt.savefig(os.path.join(output, "subject_average.png"))
plt.close()

df["average"] = df[subject_cols].mean(axis=1) top
= df.nlargest(5, "average")

plt.bar(top["name"],
top["average"])
plt.xlabel("Student")
plt.ylabel("Average")
plt.xticks(rotation=30)
plt.tight_layout()
plt.savefig(os.path.join(output, "top_performers.png"))
plt.close()
```

```
        print("cleaned_marks.csv")
        print("summary.csv")
        print("grade_distribution.png")
        print("subject_average.png")
        print("top_performers.png")

    except FileNotFoundError: print("File
        not found.") except Exception as e:
        print("Error:", e)

if name == " main ": main()
```

OUTPUT :

Riyancee Rana (12402080601119)



Riyancee Rana (12402080601119)